
📦 AWS Database Migration Service (AWS DMS)

🔹 What is AWS DMS?

AWS DMS (Database Migration Service) is a **fully managed service** that helps you **migrate databases to AWS** quickly and securely. It supports **homogeneous** (e.g., Oracle to Oracle) and **heterogeneous** (e.g., Oracle to Amazon Aurora) migrations with **minimal downtime**.

🔸 Key Features

Feature	Description
Managed Service	No need to provision servers; AWS handles availability, monitoring, and fault tolerance.
Source/Destination	Supports on-premises, EC2-hosted, and cloud-native databases.
Real-Time Replication	Uses **Change Data Capture (CDC)** to replicate ongoing changes.
Homogeneous & Heterogeneous	Supports same or different database engines.
Schema Conversion	Works with **AWS Schema Conversion Tool (AWS SCT)** for schema transformation.
Scalable and Secure	Can migrate terabytes of data securely and at scale.

🔹 Supported Databases

Source / Target Type	Examples

Relational	Oracle, MySQL, PostgreSQL, MariaDB, SQL Server, SAP ASE
NoSQL	MongoDB, Amazon DynamoDB
Cloud-Native	Amazon RDS, Aurora, Redshift
Data Warehouse	Amazon Redshift, Snowflake
S3 for flat files	Can migrate to/from S3 using CSV, Parquet

🔸 How AWS DMS Works

🛠️ Basic Workflow:

1. **Create a replication instance**
2. **Define endpoints** (source and target DBs)
3. **Configure replication tasks**:
    - Full Load
    - Ongoing Replication (CDC)
4. **Monitor using AWS Console or CloudWatch**
5. **Validate and cut over to new DB**

🧠 Types of Migrations

Type	Description
Homogeneous	Source and target use the same DB engine (e.g., MySQL → Amazon RDS MySQL)
Heterogeneous	Different engines (e.g., Oracle → Amazon Aurora); use **AWS SCT**
Continuous Replication	Real-time syncing of changes for live applications

Consolidation	Merge multiple databases into one target
Development/Test	Migrate production data to non-prod environments

---

🔒 Security

- **Encryption in transit**: SSL
- **Encryption at rest**: KMS encryption
- **VPC support** for secure networking
- **IAM roles** to manage access

💰 Pricing Overview (2025 estimate)

Component	Cost Example
Replication Instance	From ~$0.018 per hour (t3.micro) to higher for large instances
Data Transfer	Free within AWS region; standard inter-region rates apply
Storage	Included with replication instance
DMS Free Tier	Up to 750 hours per month for 1 year for t3.micro

📝 Pricing Page

✅ Use Cases

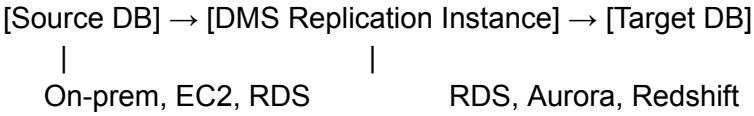
- Migrate on-premises SQL Server to Amazon RDS SQL Server
- Move Oracle workloads to Amazon Aurora PostgreSQL
- Continuous replication for disaster recovery

- Migrate production databases to dev/test environments
- Archive data from relational DBs to Amazon S3

📘 Notes for Study & Interviews

- AWS DMS uses **CDC** (Change Data Capture) for near real-time replication.
- **AWS SCT** (Schema Conversion Tool) is used for heterogeneous migrations.
- Migration is typically **one-way**, not bidirectional.
- Use **DMS Fleet Advisor** for pre-migration assessment.
- DMS can replicate to/from **S3** as a data lake or staging area.
- Ideal for **low downtime** migration scenarios.

🖼️ Diagram (Textual Representation)



Here's a **Step-by-Step Guide to Set Up a Database Migration Using AWS DMS (Database Migration Service)**:

🚀 Step-by-Step Guide: Migrate a Database Using AWS DMS

This guide assumes you are migrating from an **on-premises** or **EC2-hosted database** (like MySQL) to an **Amazon RDS** or **Aurora** instance.

🔧 Step 1: Preparation (Pre-requisites)

✅ Requirements:

- AWS Account
- Source DB (on-premises, EC2, or RDS)
- Target DB (Amazon RDS, Aurora, etc.)
- Network access between source, DMS, and target DB
- AWS SCT (if it's a **heterogeneous** migration)

📦 Step 2: Create a Target Database

- Go to the **Amazon RDS Console**
- Launch an **RDS/Aurora instance** with your desired configuration
- Ensure:
    - Proper security group (open port access from DMS)
    - Public access if needed
    - IAM role with appropriate permissions

📡 Step 3: Create a Replication Instance

1. Go to **AWS DMS Console**

2. Choose **"Replication instances" > "Create replication instance"**
3. Fill in:
    - Name
    - Instance class (e.g., `dms.t3.medium`)
    - VPC (same as DBs for best connectivity)
    - Allocated storage (depends on data size)
4. Enable **Multi-AZ** if needed for production
5. Launch and wait for status = `Available`

🌐 Step 4: Configure Endpoints

You need to create **2 endpoints**: **Source** and **Target**

🔹 Source Endpoint:

- Select **database engine** (e.g., MySQL)
- Enter:
    - Server name / IP
    - Port
    - Username / password
    - SSL mode (optional)
- Test connection (ensure DB is reachable from DMS VPC)

🔹 Target Endpoint:

- Same process (e.g., Amazon RDS PostgreSQL)
- Test the connection again

🧩 Step 5: Create a Replication Task

1. Go to **"Database migration tasks" > "Create task"**
2. Choose:
    - **Replication instance**
    - **Source and target endpoints**
    - Migration type:
        - **Migrate existing data only**
        - **Migrate existing data and replicate ongoing changes (CDC)**
        - **Replicate data changes only**
3. Table mappings:
    - Use **wildcards** or manual table selection
    - Add **transformation rules** if needed
4. Task settings:
    - Enable logging
    - Error handling (e.g., stop task on failure)

🟢 Step 6: Start the Migration Task

- Click **"Start task"**
- Monitor status (`Running`, then `Load complete`, then `CDC running` if using replication)
- Use **CloudWatch Logs** for troubleshooting
- Verify:
    - Table creation in target
    - Data consistency

🔍 Step 7: Monitor & Validate

- Check task status regularly
- Use AWS DMS **validation reports**
- Ensure row counts match on source and target
- Monitor **latency** if using CDC

🧼 Step 8: Clean Up

Once migration is verified:

- Switch application traffic to the new DB
- Stop the task
- Delete the replication instance and endpoints (optional)
- Delete old source DB (if retiring it)

📝 Tips

- Always **test in dev/staging** before production
- Consider using **DMS Fleet Advisor** for large enterprise migrations
- Combine with **AWS SCT** for schema changes (e.g., Oracle → PostgreSQL)
- For secure access:
    - Use **VPC peering**
    - Open required **ports**
    - Enable **SSL** if security is a concern

📘 Example Use Case: MySQL (EC2) → Aurora PostgreSQL

Step	Details
1	Set up target Aurora PostgreSQL in same VPC
2	Create DMS replication instance (e.g., `t3.medium`)
3	Define MySQL source and Aurora target endpoints
4	Use AWS SCT to convert schema
5	Start full load + CDC
6	Monitor until CDC latency = 0
7	Switch traffic, clean up
